

Lab02

Valdinar Pires

2027-11-08

Introdução

A coleta de dados em épocas mais recentes teve seu custo reduzido e volume ampliado significativamente. Redes sociais combinadas com dispositivos móveis rastreiam bilhões de pessoas no mundo, com informações das mais diversas origens: rotinas de passeio, amizades, históricos de compras, históricos de localização, fotografias geo-referenciadas, etc (FYI: com o escândalo da Cambridge Analytica, surgiram evidências de que o Facebook tenha registros de histórico de chamadas e SMS, para mensagem e ligações ocorridas fora do aplicativo Facebook, em telefones Android: <https://www.theverge.com/2018/3/25/17160944/facebook-call-history-sms-data-collection-android>). Empresas, por meio de programas de fidelidade, aprendem hábitos de compras, propensão de compras de certos produtos dependendo de horários, produtos que são comprados juntos e informações financeiras referentes a pagamentos. Desta maneira, o volume das bases de dados cresce vertiginosamente, de modo que é necessária a mudança de paradigma na estatística moderna: os dados não mais podem trafegar até o analista de dados; a análise deve ser transportada até os dados.

Objetivos Ao fim deste laboratório, você deve ser capaz de:

- Importar um arquivo volumoso por partes;
- Calcular estatísticas suficientes para métrica de interesse em cada uma das partes importadas;
- Manter em memória apenas o conjunto de estatísticas suficientes e utilizar a memória remanescente para execução de cálculos;
- Combinar as estatísticas suficientes de modo a compor a métrica de interesse.

Lendo 100.000 observações por vez (se você acredita que seu computador não tem memória suficiente, utilize 10.000 observações por vez), determine o percentual de vôos por Cia. Aérea que apresentou atraso na chegada (`ARRIVAL_DELAY`) superior a 10 minutos. As companhias a serem utilizadas são: AA, DL, UA e US. A estatística de interesse deve ser calculada para cada um dos dias de 2015. Para a determinação deste percentual de atrasos, apenas verbos do pacote `dplyr` e comandos de importação do pacote `readr` podem ser utilizados. Os resultados para cada Cia. Aérea devem ser apresentados em um formato de calendário.

Observação: a atividade descrita no slide pede para ler apenas 100 registros por vez. Aqui, mudamos para 100 mil registros por vez, para que haja melhor aproveitamento do tempo em classe.

Instruções

1- Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

As estatísticas suficientes são o número total de voos e o número de voos que atrasaram mais de 10 minutos (`ARRIVAL_DELAY > 10`), para cada grupo de dia, mês e companhia aérea.

2- Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):

```
library(tidyverse)
```

Warning: package 'tidyverse' was built under R version 4.4.3

Warning: package 'ggplot2' was built under R version 4.4.3

Warning: package 'tibble' was built under R version 4.4.3

Warning: package 'tidyr' was built under R version 4.4.3

Warning: package 'readr' was built under R version 4.4.3

Warning: package 'purrr' was built under R version 4.4.3

Warning: package 'dplyr' was built under R version 4.4.3

Warning: package 'stringr' was built under R version 4.4.3

Warning: package 'forcats' was built under R version 4.4.3

Warning: package 'lubridate' was built under R version 4.4.3

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.1
v ggplot2    4.0.0      v tibble     3.2.1
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.0.4

-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
getStats = function(input, pos){

# a. Filtrar as companhias e o ano
input %>%
  filter(
    AIRLINE %in% c("AA", "DL", "UA", "US"),
    YEAR == 2015
  ) %>%

# b. Remover valores faltantes
drop_na(YEAR, MONTH, DAY, AIRLINE, ARRIVAL_DELAY) %>%

# c. Agrupar por ano, dia, mês e companhia
group_by(YEAR, DAY, MONTH, AIRLINE) %>%

# d. Calcular as estatísticas suficientes
summarise(
  n_parcial_voos = n(),
  n_parcial_atrasos = sum(ARRIVAL_DELAY > 10),
  .groups = "drop"
)
}
```

3- Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

```
#Lendo o arquivo de interesse
teste <- read_csv_chunked("flights.csv.zip",
  #transformar os dados em o que ?
  callback= DataframeCallback$new(getStats),
  #qual a quantidade de leitura por vez?
  chunk_size = 100000,
```

```

#qual coluna ler e qual classe de variavel agregar?
col_types = cols_only( YEAR = 'i', MONTH = 'i',
                        DAY = 'i', AIRLINE = 'c',
                        ARRIVAL_DELAY = 'i' )

#Observando dias especificos do ano)
quasela <- teste %>% filter(AIRLINE == "AA", DAY == 7, MONTH == 3)

```

4 - Crie uma função chamada computeStats que:

```

#Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de

computeStats <- function(input) {

  input %>%
    group_by(AIRLINE, DAY, MONTH, YEAR) %>%
    summarise(
      Perc = sum(n_parcial_atrasos) / sum(n_parcial_voos),
      .groups = "drop"
    ) %>%
  #Retorne as informações em um tibble contendo apenas as seguintes colunas com CIA, DATA
    rename(
      Cia = AIRLINE
    ) %>%
    mutate(
      Data = as.Date(paste(YEAR, MONTH, DAY, sep = "-"))
    ) %>%
    select(Cia, Data, Perc)
}

stats <- computeStats(teste)

```

5 - Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

```

library(ggcal)
library(ggplot2)

pal <- scale_fill_gradient(
  low = "#4575b4",
  high = "#d73027"
)

```

```
baseCalendario <- function(stats, cia) {

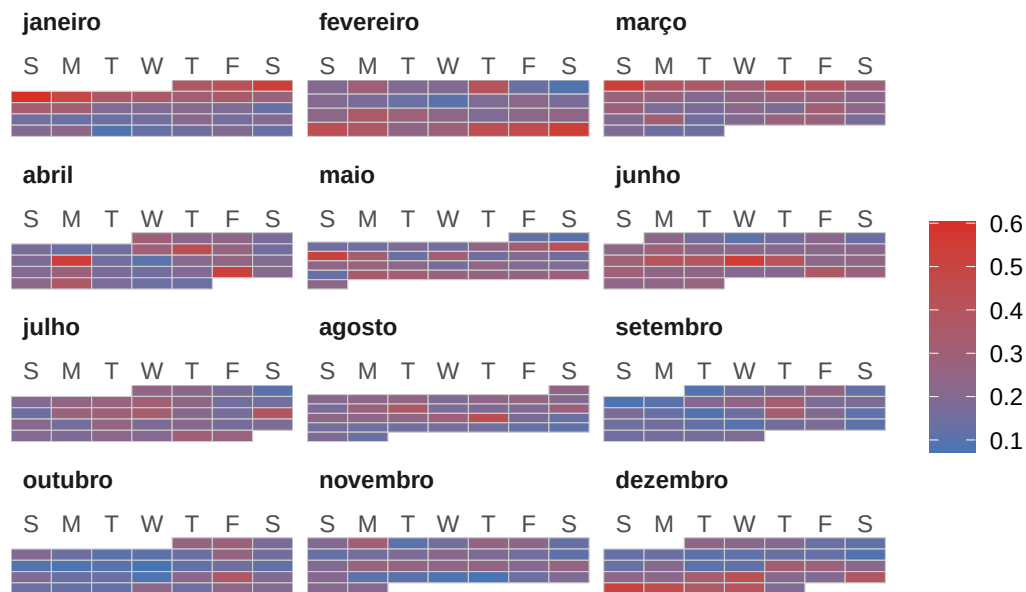
  dados <- stats %>%
    filter(Cia == cia)

  ggcal(
    dates = dados$Data,
    fills = dados$Perc
  )
}

cAA <- baseCalendario(stats, "AA")
cDL <- baseCalendario(stats, "DL")
cUA <- baseCalendario(stats, "UA")
cUS <- baseCalendario(stats, "US")

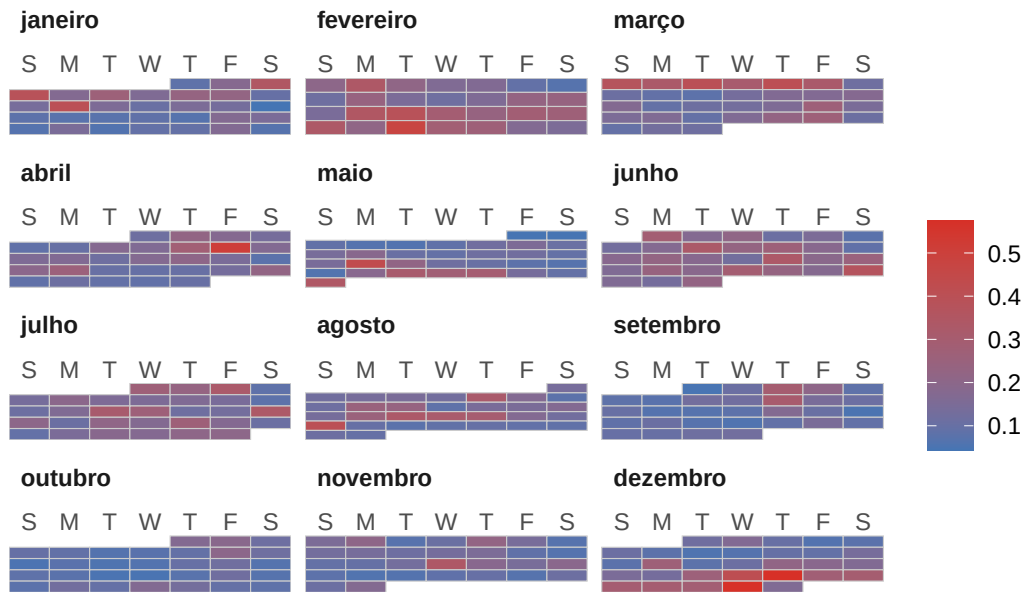
cAA + pal + ggtitle("AA")
```

AA



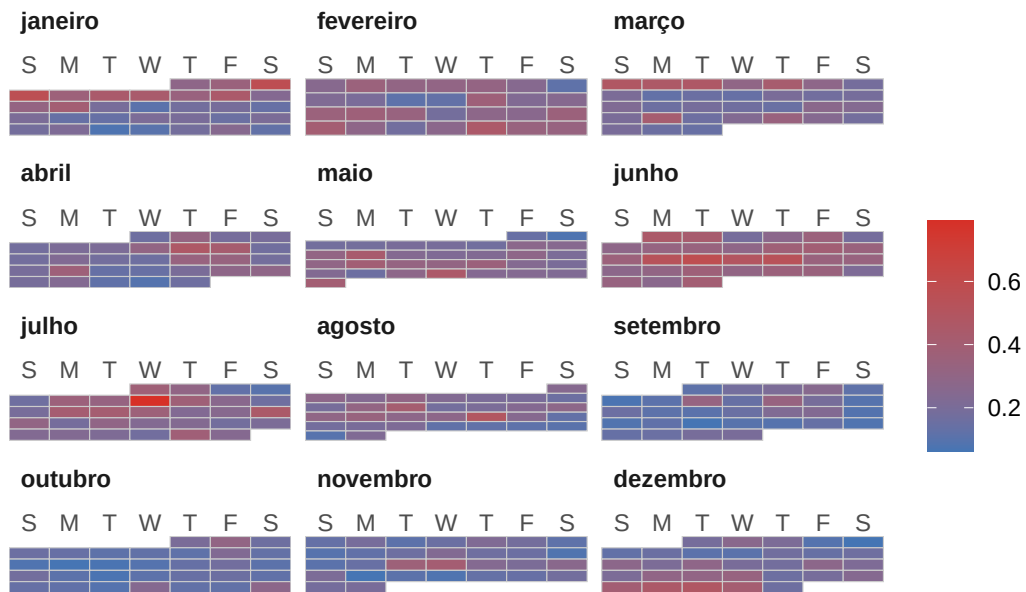
```
cDL + pal + ggtitle("DL")
```

DL



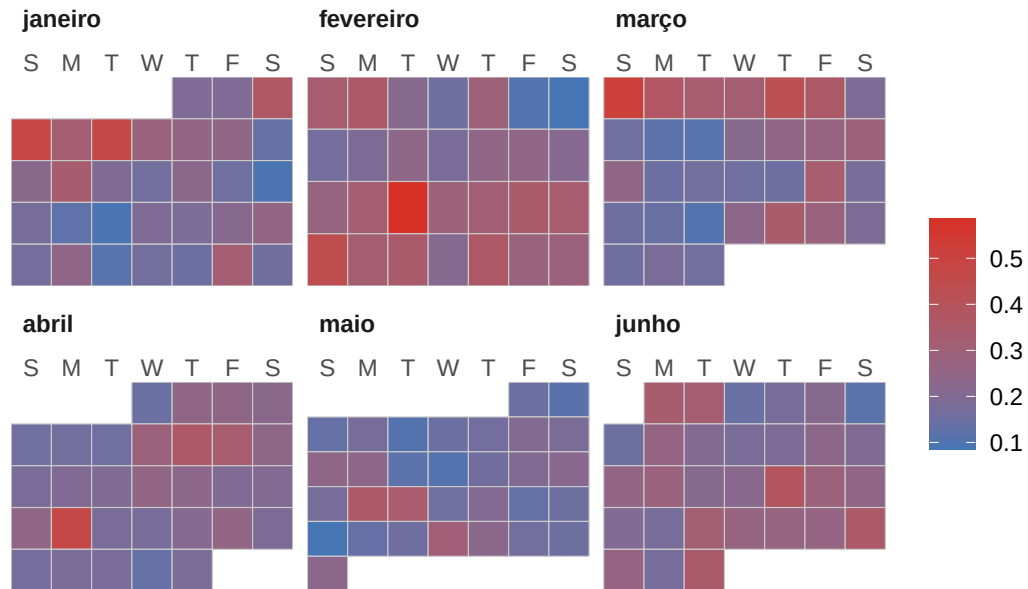
```
cUA + pal + ggtitle("UA")
```

UA



```
cUS + pal + ggtitle("US")
```

US



```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-08-24 22:27:32 -03"
```